Discussion 8: GUI and File System

Discussion Topic:

In your initial post, explain how the file system in GUI environments acts as the Application Programming Interface (API) that the OS provides for accessing bits on the hard drive.
Describe the process, including the points at which the OS has to switch from user mode to kernel mode. You can do so for computer systems in general or a specific OS of your choosing.
Compare this with the way that command line environments perform the same operations.

My Post:

Hello Class

Operating systems (OSs) manage files on the hard drive using file systems that function similarly to an API but for storage access. In a GUI environment, this translates to the user seeing folders, icons, drag-and-drop actions, and file names. However, these visual elements are not accessing the disk itself; they translate user actions into file-system requests that the OS can process. In other words, the elements are an interface layered above the file system, and the file system is an abstraction layer between the hard disk and the GUI.

For example, when a user double-clicks a document, the GUI, such as Windows File Explorer, runs in user mode. It asks the OS to open the file by using file-system services. Windows uses a file system that implements functions such as `CreateFile`, `ReadFile`, and `WriteFile` to create or open files and read or write data (Microsoft, 2025a, 2025b, 2025c). To perform the actual file operation, the CPU must switch from user mode to kernel mode to process the GUI request safely. This switch between user mode and kernel mode is necessary for security and system stability because applications should not directly access the disk, file-system metadata, or storage drivers. Windows documentation explains that the File Explorer GUI runs in user mode, while the file-system service runs in kernel mode (Microsoft, 2025d).

In kernel mode, the file-system service can check permissions, resolve the path, consult file-system metadata, and send I/O requests through the kernel I/O manager and storage drivers. The Windows kernel-mode I/O manager is the Windows service that manages communication between applications and device driver interfaces, often using I/O request packets (Microsoft, 2025e).
Linux uses different naming conventions from Windows, but it follows the same basic idea. Files and Dolphin are examples of GUI file managers used in Linux desktop environments. These GUIs rely on the same kernel file-system layer that is also used by command-line tools. The Linux Virtual File System is a uniform file-system interface to user-space programs that allows different file-system implementations to exist in the kernel (Linux kernel developers, n.d.). For example, when a Linux process needs to open a file, it calls the `open()` function. The Linux kernel then returns a file descriptor that later system calls, such as `read()`, `write()`, and `lseek()`, use to access or move through the file (Kerrisk, n.d.).

The command line can also be used to access, create, copy, move, rename, read, write, and delete files. It performs the same operations as a GUI file manager, but with less “visual wrapping.”
For example, the operations `cat notes.txt`, `cp file1 file2`, `mv oldname newname`, or `rm draft.txt` are user-typed commands that run in user mode. The command line still has to request file operations from the OS through system calls; it is a user interface that relies on the file-system layer.

In conclusion, the difference between GUIs and command lines is mainly that a GUI uses icons, windows, menus, and drag-and-drop, whereas a command line uses text commands and arguments. In other words, both interfaces run in user mode and rely on the same kernel file system to access or modify files on a hard drive, regardless of the OS used.

-Alex

References:

Kerrisk, M. (n.d.). open(2): Linux manual page. man7.org. https://man7.org/linux/man-pages/man2/open.2.htm

Linux kernel developers. (n.d.). Overview of the Linux Virtual File System. The Linux Kernel Documentation. https://docs.kernel.org/filesystems/vfs.html

Microsoft. (2025a, February 8). CreateFileA function. Microsoft Learn. https://learn.microsoft.com/en-us/windows/win32/api/fileapi/nf-fileapi-createfilea

Microsoft. (2025b, July 22). ReadFile function. Microsoft Learn. https://learn.microsoft.com/en-us/windows/win32/api/fileapi/nf-fileapi-readfile

Microsoft. (2025c, March 3). WriteFile function. Microsoft Learn. https://learn.microsoft.com/en-us/windows/win32/api/fileapi/nf-fileapi-writefile

Microsoft. (2025d, October 28). User mode and kernel mode. Microsoft Learn. https://learn.microsoft.com/en-us/windows-hardware/drivers/gettingstarted/user-mode-and-kernel-mode

Microsoft. (2025e, May 12). Windows kernel-mode I/O manager. Microsoft Learn. https://learn.microsoft.com/en-us/windows-hardware/drivers/kernel/windows-kernel-mode-i-o-manager
-Alex

References:

Kerrisk, M. (n.d.). open(2): Linux manual page. man7.org. https://man7.org/linux/man-pages/man2/open.2.htm

Linux kernel developers. (n.d.). *Overview of the Linux Virtual File System*. The Linux Kernel Documentation. https://docs.kernel.org/filesystems/vfs.html

Microsoft. (2025a, February 8). *CreateFileA function*. Microsoft Learn. https://learn.microsoft.com/en-us/windows/win32/api/fileapi/nf-fileapi-createfilea

Microsoft. (2025b, July 22). *ReadFile function*. Microsoft Learn. https://learn.microsoft.com/en-us/windows/win32/api/fileapi/nf-fileapi-readfile

Microsoft. (2025c, March 3). *WriteFile function*. Microsoft Learn. https://learn.microsoft.com/en-us/windows/win32/api/fileapi/nf-fileapi-writefile

Microsoft. (2025d, October 28). *User mode and kernel mode*. Microsoft Learn. https://learn.microsoft.com/en-us/windows-hardware/drivers/gettingstarted/user-mode-and-kernel-mode

Microsoft. (2025e, May 12). *Windows kernel-mode I/O manager*. Microsoft Learn. https://learn.microsoft.com/en-us/windows-hardware/drivers/kernel/windows-kernel-mode-i-o-manager